

2023년 3회 정보처리기사 실기 복원 문제 (해설 추가)

출처: <https://chobopark.tistory.com/453> (Life-Journey)

1번

다음은 Java 코드이다. 올바른 출력 결과를 작성하시오.

```
public class main {
    public static void main(String[] args) {
        A b = new B();
        b.paint();
        b.draw();
    }
}

class A {
    public void paint() {
        System.out.print("A");
        draw();
    }
    public void draw() {
        System.out.print("B");
        draw();
    }
}

class B extends A {
    public void paint() {
        super.draw();
        System.out.print("C");
        this.draw();
    }
    public void draw() {
        System.out.print("D");
    }
}
```

정답: BDCDD

해설:
오버라이딩(메서드 재정의)과 **super** / **this**의 차이를 묻는 어려운 문제입니다.

핵심 원리 - 오버라이딩된 메서드는 "**실제 객체(B)**" 기준으로 실행됩니다.

- A b = new B(); → 변수 타입은 A지만, 실제 객체는 **B**입니다.
- 그래서 일반적으로 b.draw() 를 부르면 **B의 draw()**가 실행됩니다.
- 단, **super.draw()** 는 **강제로 부모(A)의 메서드**를 부릅니다.

- b.paint() 실행 (B에서 오버라이딩됨 → B의 paint):
1. super.draw() → **부모 A의 draw()** 실행 → "B" 출력. 그런데 A의 draw() 안에 또 draw() 가 있습니다. 이때의 draw()는 실제 객체 B 기준이라 **B의 draw()**가 불립니다 → "D" 출력. (B의 draw는 재귀 없이 끝남)
 - 여기까지 출력: **B D**
 2. System.out.print("C") → "C" 출력 → **B D C**
 3. this.draw() → 실제 객체 B의 draw() → "D" 출력 → **B D C D**
- b.draw() 실행 → B의 draw() → "D" 출력 → 최종 **B D C D D = BDCDD**

핵심: "**super.메서드 = 무조건 부모 것**", "그 외 메서드 호출 = 실제 객체(B) 것". A의 draw 안에서 부른 draw()도 객체가 B이므로 B의 draw가 불린다는 게 함정입니다.

2번

다음 설명하는 용어를 보기에 맞게 골라 기호를 작성하시오.

사용자 자격 증명을 공유하지 않고도 타사 애플리케이션이 데이터·리소스에 액세스할 수 있도록 권한을 부여하는 개방형 표준 프로토콜. 액세스 토큰을 사용하며, 비밀번호는 타사와 공유되지 않는다. 소셜 미디어 플랫폼(Facebook, Twitter 등)에서 널리 사용된다.

보기: ㄱ. NFT ㄴ. JWT ㄷ. SAML ㄹ. accessToken ㄴ. SSO ㅂ. OIDC ㅅ. Authentication ㅇ. OAuth ㅈ. Cookie ㅊ. Refresh Token

정답: ㅇ (OAuth)

해설:
OAuth는 "비밀번호를 알려주지 않고도, 다른 서비스에 일부 권한만 위임"하는 **인가(Authorization)** 표준 프로토콜입니다.

- 예: 어떤 앱에 "구글 계정으로 로그인"할 때, 구글 비밀번호를 그 앱에 직접 주지 않습니다. 대신 구글이 발급한 **액세스 토큰(Access Token)**으로 제한된 권한만 넘겨줍니다.
- 핵심: **비밀번호 공유 X, 토큰으로 권한 위임, 소셜 로그인**의 기반 기술.
- 주의: OAuth는 "인증(누구인지 확인)"이 아니라 "**인가(권한 부여)**"에 초점이 있습니다.

비교 정리:
- **SSO**: 한 번 로그인으로 여러 서비스 이용(통합 로그인)
- **SAML**: 인증 정보를 XML로 교환하는 표준(주로 기업/SSO)
- **OAuth**: 권한 위임(소셜 로그인의 토대)

3번

리눅스에서 사용자에게 읽기/쓰기/실행 권한을, 그룹에게 읽기/실행을, 그 외에는 실행 권한을 test.txt 파일에 부여하는 명령어이다. 빈칸을 작성하시오. (8진법 사용)

((1)) ((2)) test.txt

정답: (1) chmod / (2) 751

해설:
리눅스 **파일 권한 설정** 문제입니다. 명령어는 **chmod**(change mode)이고, 권한은 **8진수 숫자 3자리**로 표현합니다.

권한 종류와 숫자 값:
- **읽기(r) = 4**
- **쓰기(w) = 2**
- **실행(x) = 1**

각 자리는 순서대로 **소유자(User) / 그룹(Group) / 기타(Other)**의 권한이며, 해당 권한 값을 더합니다.

- 사용자(소유자): 읽기+쓰기+실행 = 4+2+1 = **7**
- 그룹: 읽기+실행 = 4+1 = **5**
- 기타: 실행 = **1**

따라서 **chmod 751 test.txt**.

암기 팁: "**읽기4-쓰기2-실행1**, 세 자리는 **소유자-그룹-기타**". 자주 나오는 패턴: rwx=7, rw-=6, r-x=5, r--=4.

4번

다음은 C 언어 코드이다. 알맞는 출력 결과를 작성하시오.

```
#include <stdio.h>

int test(int n) {
    int i, sum = 0;
    for (i = 1; i <= n / 2; i++){
        if (n % i == 0)
            sum += i;
    }
}
```

```

        if (n == sum)
            return 1;
        return 0;
    }

    int main(){
        int i, sum = 0;
        for (i = 2; i <= 100; i++){
            if (test(i))
                sum += i;
        }
        printf("%d ", sum);
        return 0;
    }

```

정답: 34

해설:

이 코드는 **완전수(Perfect Number)**를 찾는 프로그램입니다.

- **완전수**란? 자기 자신을 뺀 **약수들의 합이 자기 자신과 같은** 수입니다.
- `test(n)`: n의 약수(n/2 이하)를 모두 더해서, 그 합이 n과 같으면 완전수(1 반환).

100 이하의 완전수:

- **6** = 1 + 2 + 3 (약수 1,2,3의 합) ✓

- **28** = 1 + 2 + 4 + 7 + 14 ✓

main에서는 2~100의 완전수를 모두 더합니다 → 6 + 28 = **34**

핵심: 코드를 보고 "이게 무슨 알고리즘인지" 패턴을 알아채는 것이 중요합니다. "약수의 합 == 자기 자신"은 **완전수** 판별의 전형적 코드입니다.

5번

C언어에서 구조체의 멤버에 접근하기 위해 괄호 안의 기호를 작성하시오. (출력결과: 10 / 10)

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Data{
    char c;
    int *numPtr;
} Data;

int main(){
    int num = 10;
    Data d1;
    Data *d2 = malloc(sizeof(struct Data));

    d1.numPtr = &num;
    d2 ( ) numPtr = &num;

    printf("%d\n", *d1.numPtr);
    printf("%d\n", *d2 ( ) numPtr);

    free(d2);
    return 0;
}

```

정답: -> (화살표)

해설:

C언어에서 **구조체 멤버에 접근하는 두 가지 방법**을 구분하는 문제입니다.

- **점(.) 연산자**: 구조체 변수가 직접 있을 때 사용. 예: `d1.numPtr`
- **화살표(->) 연산자**: 구조체 포인터일 때 사용. 예: `d2->numPtr`

d2 는 Data *d2 로 선언된 **포인터**입니다(malloc으로 메모리를 동적 할당받음). 포인터를 통해 멤버에 접근할 때는 점(.)이 아니라 **화살표(->)**를 써야 합니다.

참고: d2->numPtr 은 사실 (*d2).numPtr 과 같은 의미입니다. 화살표는 이걸 간단하게 쓴 것입니다.

암기 팁: "구조체 변수는 점(.), 구조체 포인터는 화살표(->)".

6번

다음 빈칸에 들어갈 UNION 연산의 결과값을 작성하시오.

T1: 3, 1, 4 / T2: 2, 4, 3

```
SELECT A FROM T1
UNION
SELECT A FROM T2
ORDER BY A DESC;
```

정답: 4 / 3 / 2 / 1

해설:

집합 연산자 UNION과 정렬 ORDER BY를 묻는 문제입니다.

- **UNION:** 두 결과를 합치되, 중복은 제거합니다. (중복까지 모두 합치려면 UNION ALL 을 씁니다.)
- T1 = {3, 1, 4}, T2 = {2, 4, 3}을 합치면: 3, 1, 4, 2, 4, 3 → **중복 제거** → {1, 2, 3, 4}
- **ORDER BY A DESC:** A를 기준으로 **내림차순(큰 값부터)** 정렬 → 4, 3, 2, 1

따라서 결과: 4 / 3 / 2 / 1

암기 팁: "UNION = 중복 제거 합집합, UNION ALL = 중복 포함", "DESC = 내림차순, ASC = 오름차순(기본값)".

7번

다음 설명은 서버 접근 통제 유형이다. 괄호 안에 들어갈 용어를 영어 약자로 작성하시오.

- (1) 규칙 기반(Rule-Based) 접근통제 정책. 객체의 허용 등급과 주체의 접근 허가 권한에 근거하여 접근 제한.
- (2) 중앙 관리자가 통제하며 조직 내 역할(Role)에 기초하여 자원 접근 제한.
- (3) 신분 기반(Identity-Based) 접근통제 정책. 주체나 그룹의 신분에 근거하여 접근 제한.

정답: (1) MAC / (2) RBAC / (3) DAC

해설:

접근 통제 3가지 방식은 시험 단골입니다.

- **MAC(Mandatory Access Control, 강제적 접근 통제):** 중앙(관리자)이 정한 보안 등급(규칙)으로 통제. 사용자가 임의로 바꿀 수 없습니다. 군사·국가 기밀 처럼 보안이 매우 중요한 곳에 사용. 키워드: "규칙 기반, 보안 등급".
- **RBAC(Role-Based Access Control, 역할 기반 접근 통제):** 사람이 아니라 **역할(Role)**에 권한을 부여. 예: "팀장 역할"에 권한을 묶어두고 사람을 역할에 배정. 관리가 편리. 키워드: "역할(Role)".
- **DAC(Discretionary Access Control, 임의적 접근 통제):** **자원의 소유자(주인)**가 권한을 자유롭게 줬다 뺐었다 할 수 있음. **신분 기반.** 유연하지만 보안성은 낮음. 키워드: "소유자 임의 결정, 신분 기반".

암기 팁: "MAC=강제(등급), RBAC=역할, DAC=소유자 마음대로(임의)".

8번

다음 C언어 코드에 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int f(int n) {
    if(n <= 1) return 1;
    else return n * f(n-1);
}
```

```
int main() {
    printf("%d", f(7));
}
```

정답: 5040

해설:
재귀 함수(자기 자신을 다시 부르는 함수)로 팩토리얼(계승)을 구하는 코드입니다.

- 팩토리얼 $n! = n \times (n-1) \times (n-2) \times \dots \times 1$
- $f(n) = n * f(n-1)$ 은 정확히 이 정의입니다.
- 종료 조건: $n \leq 1$ 이면 1을 반환 (더 이상 재귀하지 않고 멈춤).

계산:
- $f(7) = 7 \times f(6) = 7 \times 6 \times f(5) = \dots = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$
- $= 7! = \mathbf{5040}$

핵심: 재귀 함수는 반드시 종료 조건이 있어야 무한 반복을 막을 수 있습니다. 그리고 가장 작은 경우(여기선 $n=1$)부터 거슬러 올라가며 계산한다고 이해하면 쉽습니다.

9번

다음 설명에 대해 괄호 안에 알맞는 용어를 영어 약자로 작성하시오.

- 자료를 일정한 크기로 정하여 순서대로 전송하는 자료 전송 방식
- 셀(고정 길이 53바이트 패킷)을 이용하여 처리가 단순하고 고속망에 적합
- 가상 채널 기반의 연결 지향 서비스, 통계적 다중화 방식 이용

정답: ATM

해설:
ATM(Asynchronous Transfer Mode, 비동기 전송 모드)은 데이터를 53바이트 고정 크기의 "셀(Cell)" 단위로 잘라 전송하는 방식입니다.

- 셀 크기가 고정(53바이트 = 헤더 5 + 데이터 48)이라 처리 속도가 빠르고 예측 가능합니다.
- 가상 채널(가상 회선) 기반의 연결 지향형 서비스입니다(앞서 배운 가상회선 방식).
- 고속 데이터·음성·영상을 함께 전송하는 고속망에 적합합니다.

핵심 키워드: "53바이트 고정 셀"이 나오면 ATM입니다.

(주의: 여기서 ATM은 은행 현금인출기가 아니라 통신 방식입니다.)

10번

다음은 C언어의 포인터 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int main() {
    char* p = "KOREA";
    printf("%s\n", p);
    printf("%s\n", p+1);
    printf("%c\n", *p);
    printf("%c\n", *(p+3));
    printf("%c\n", *p+4);
}
```

정답: KOREA / OREA / K / E / O

해설:
포인터로 문자열을 다루는 문제로, %s / %c 의 차이와 포인터 연산이 핵심입니다.

문자열 "KOREA"의 각 글자 위치:

K(p) O(p+1) R(p+2) E(p+3) A(p+4)

- `printf("%s\n", p)` → p부터 끝까지 문자열 → **KOREA**
- `printf("%s\n", p+1)` → p+1(O 위치)부터 끝까지 → **OREA**
- `printf("%c\n", *p)` → p가 가리키는 한 글자 → **K**
- `printf("%c\n", *(p+3))` → p+3(E 위치)의 한 글자 → **E**
- `printf("%c\n", *p+4)` → 여기가 함정! *p 는 'K'인데, 문자 'K' + 4를 계산합니다. 'K'의 아스키코드는 75, 75+4=79, 아스키 79는 'O'. → **O**

핵심: `*(p+4)` (주소에 4를 더함)와 `*p+4` (값에 4를 더함)는 **완전히 다릅니다**. 괄호 위치를 잘 봐야 합니다. 그리고 문자에 정수를 더하면 아스키코드로 계산됩니다.

11번

다음은 Java 코드에 대한 알맞는 출력값을 작성하시오.

```
class Parent {
    int compute(int num) {
        if(num <= 1)
            return num;
        return compute(num-1) + compute(num-2);
    }
}

class Child extends Parent {
    int compute(int num) {
        if(num <= 1)
            return num;
        return compute(num-1) + compute(num-3);
    }
}

public class main {
    public static void main(String args[]) {
        Parent obj = new Child();
        System.out.print(obj.compute(7));
    }
}
```

정답: 2

해설:

오버라이딩 + 재귀가 결합된 문제입니다.

- `Parent obj = new Child();` → 실제 객체는 **Child**이므로, `obj.compute(7)` 은 **Child의 compute()**가 실행됩니다.
- Child의 compute 안에서 `compute(...)` 를 다시 부를 때도, 객체가 Child이므로 계속 **Child의 compute**가 호출됩니다.
- 따라서 모든 재귀는 `compute(num-1) + compute(num-3)` 규칙을 따릅니다.

계산 (num<=1이면 num 반환):

- `compute(0)=0, compute(1)=1`
- `compute(2) = compute(1) + compute(-1) = 1 + (-1) = 0`
- `compute(3) = compute(2) + compute(0) = 0 + 0 = 0`
- `compute(4) = compute(3) + compute(1) = 0 + 1 = 1`
- `compute(5) = compute(4) + compute(2) = 1 + 0 = 1`
- `compute(6) = compute(5) + compute(3) = 1 + 0 = 1`
- `compute(7) = compute(6) + compute(4) = 1 + 1 = 2`

핵심: 부모 타입 변수라도 **실제 객체(Child)의 오버라이딩 메서드**가 실행됩니다. 재귀는 작은 값부터 표를 그려 차근차근 계산하면 실수가 적습니다.

12번

IP 패킷에서 외부의 공인 IP 주소와 포트 주소에 해당하는 내부 IP 주소를 재기록하여 라우터를 통해 네트워크 트래픽을 주고받는 기술은 무엇인가?

정답: NAT (Network Address Translation)

해설:

NAT(Network Address Translation, 네트워크 주소 변환)는 사실 IP ↔ 공인 IP를 서로 변환해주는 기술입니다.

- 우리가 집에서 쓰는 여러 기기(폰, PC, TV)는 각각 **사설 IP**(예: 192.168.x.x)를 가집니다. 이 사설 IP는 인터넷에서 직접 통신할 수 없습니다.
- NAT는 이 사설 IP를 공유기가 가진 **하나의 공인 IP**로 바꿔서 인터넷과 통신하게 해줍니다.
- 장점: 부족한 공인 IP 주소를 절약하고, 내부 IP를 숨겨 보안에도 도움이 됩니다.

비유: 회사 대표 전화번호(공인 IP) 하나로 외부와 통화하고, 내부 직원들은 내선번호(사설 IP)로 구분하는 것과 비슷합니다.

13번

다음 자바 코드를 실행할 경우 에러가 발생합니다. 에러가 발생하는 라인명을 작성하시오.

```
class Person {
    private String name;
    public Person(String val) {
        name = val;
    }
    public static String get() {    // ← 7번 라인: static에서 인스턴스 멤버 name 참조 → 에러
        return name;
    }
    public void print() {
        System.out.println(name);
    }
}

public class main {
    public static void main(String[] args) {
        Person obj = new Person("Kim");
        obj.print();
    }
}
```

정답: 7

해설:

static 메서드 안에서 인스턴스 변수를 직접 쓸 수 없다는 규칙을 묻는 문제입니다.

- 인스턴스 변수(name):** 객체를 만들어야 생기는 변수입니다.
- static 메서드(get):** 객체 없이도 호출되는 메서드입니다. 즉 "객체가 없는 상황"에서도 실행될 수 있습니다.
- 따라서 static 메서드 안에서 객체에 속한 name 을 참조하면, "**어느 객체의 name인지 알 수 없어**" 컴파일 에러가 납니다.

7번 라인 return name; 에서 static 메서드가 인스턴스 변수 name을 참조하므로 에러가 발생합니다.

핵심 규칙: "**static 영역에서는 non-static(인스턴스) 멤버를 직접 사용할 수 없다**". 사용하려면 객체를 먼저 만들어 객체.name 형태로 접근해야 합니다.

14번

다음은 파이썬에 대한 문제이다. 밑줄친 부분에 알맞는 답을 작성하시오. (입력값 2와 3, 출력: 2 3 / 2 + 3 = 5)

```
print("파이썬 입출력에 대한 문제입니다.")

num1, num2 = input()._____()
num1 = int(num1)
num2 = int(num2)
print(num1, num2)

num3 = num1 + num2
print(num1 + " + " + num2 + " = " + num3)
```

정답: split

해설:

파이썬에서 한 줄에 여러 값을 입력받을 때 쓰는 **split() 메서드** 문제입니다.

- `input()` 은 입력 한 줄을 **하나의 문자열**로 받습니다. 예: "2 3" (공백 포함 문자열)
- **split()** : 문자열을 **공백 기준으로 잘라** 여러 조각의 리스트로 만듭니다. "2 3".split() → ['2', '3']
- `num1, num2 = "2 3".split()` → `num1='2', num2='3'` (각각 변수에 나눠 담김)

이렇게 두 값을 한 번에 받을 수 있습니다. (그 후 `int()` 로 숫자로 변환)

핵심: "**한 줄에 여러 입력 → split()으로 쪼갬다**"가 파이썬 입력의 기본 패턴입니다.

(참고: 마지막 print문은 문자열 num1과 숫자 num3을 **+** 로 합쳐 실재론 오류가 나지만, 문제는 split 부분만 묻습니다.)

15번

다음은 판매와 관련된 다이어그램이다. 해당 다이어그램의 명칭을 쓰시오. (※ 원문에 다이어그램 이미지 포함)

정답: 패키지

해설:

UML 패키지 다이어그램(Package Diagram)을 묻는 문제입니다.

- **패키지(Package)**란 관련 있는 클래스나 요소들을 **폴더처럼 묶은 단위**입니다. 다이어그램에서 **폴더 모양** 도형으로 표현됩니다.
- 패키지 다이어그램은 이 패키지들 사이의 **의존 관계(import, access 등)**를 보여주는 **구조 다이어그램**입니다.
- 시스템이 커지면 클래스가 너무 많아지므로, 비슷한 것끼리 패키지로 묶어 전체 구조를 한눈에 보기 쉽게 합니다.

UML 다이어그램은 크게 **구조 다이어그램**(클래스, 객체, 컴포넌트, 패키지, 배치 등)과 **행위 다이어그램**(유스케이스, 시퀀스, 활동, 상태 등)으로 나뉩니다. **폴더 모양 = 패키지 다이어그램**을 기억하세요.

16번

다음 설명에 알맞는 답을 보기에서 골라 작성하시오.

- 입력값과 출력값이 특정 그룹으로 분류되며, 분류된 그룹 내 값들이 동일하게 취급된다는 특성을 이용하는 기법
- 하나의 값은 그 분할 내 모든 값을 대표
- 모든 입력값을 시험할 필요가 없게 하여 효율적 수행
- 일반적으로 경계값 분석기법과 함께 테스트 케이스 도출에 사용

보기: ㉠. Equivalence Partitioning ㉡. Statement Coverage ㉢. Boundary Value Analysis ㉣. Branch Coverage ㉤. Error Guessing ㉥. Condition Coverage ㉦. Cause Effect Graph

정답: ㉠ (Equivalence Partitioning, 동치 분할)

해설:

동치 분할(Equivalence Partitioning)은 블랙박스 테스트 기법입니다.

- 블랙박스 테스트는 코드 내부를 모르고, **입력과 출력만 보고** 테스트하는 방식입니다.
- **동치 분할:** 입력값을 **비슷하게 동작할 그룹(동치 클래스)**으로 나누고, **각 그룹에서 대표값 하나씩만** 테스트합니다. 같은 그룹이면 하나만 테스트해도 충분하다고 보는 것입니다.
- 예: "1~100을 입력"이면, 유효 그룹(1~100)과 무효 그룹(0 이하, 101 이상)으로 나눠 대표값만 시험합니다.

대표적 블랙박스 기법:

- **동치 분할:** 그룹별 대표값 테스트
- **경계값 분석(Boundary Value Analysis):** 경계(끝값, 예: 0, 1, 100, 101)에서 오류가 잘 나므로 경계를 집중 테스트
- **원인-효과 그래프, 오류 추측 등**

키워드 "**그룹으로 분류, 대표값**"은 **동치 분할**입니다.

17번

다음은 클라우드에 대한 유형 문제이다. 괄호 안에 알맞는 답을 보기에서 골라 작성하시오. (※ 원문에 이미지 포함)

보기: PaaS, SaaS, IaaS

정답: (1) IaaS / (2) PaaS / (3) SaaS

해설:
클라우드 서비스 3가지 모델을 묻는 문제입니다. 제공 범위(빌려주는 양)에 따라 나뉩니다.

- **IaaS(Infrastructure as a Service): 인프라(서버, 저장소, 네트워크)**만 빌려줍니다. 운영체제부터 위는 사용자가 직접 설치·관리합니다. 가장 기초 단계. 예: AWS EC2.
- **PaaS(Platform as a Service): 인프라 + 개발 플랫폼(OS, 실행환경)**까지 빌려줍니다. 개발자는 코드만 올리면 됩니다. 예: Google App Engine, Heroku.
- **SaaS(Software as a Service): 완성된 소프트웨어**를 바로 사용합니다. 설치 없이 웹으로 이용. 예: Gmail, Google Docs, Netflix.

암기 팁: "IaaS(인프라만) → PaaS(플랫폼까지) → SaaS(완제품)" 순으로 빌려주는 양이 많아집니다.

피자 비유로 자주 설명합니다: IaaS는 재료만, PaaS는 반조리, SaaS는 완성된 피자.

18번

다음은 프로토콜 종류에 관한 설명이다. 알맞는 답을 작성하시오.

- 최소 Hop count를 파악하여 라우팅하는 프로토콜
- 거리와 방향으로 길을 찾아가는 Distance Vector 다이나믹 프로토콜
- Routing Table에 인접 라우터 정보를 저장하여 경로 결정
- 최대 Hop count는 15로 거리가 짧아 IGP로 많이 이용

정답: RIP

해설:
RIP(Routing Information Protocol)는 가장 오래되고 단순한 라우팅 프로토콜입니다.

- **Hop count(홉 수):** 목적지까지 거쳐가는 라우터의 개수입니다. RIP는 홉 수가 가장 적은 경로를 최단 경로로 선택합니다.
- **거리 벡터(Distance Vector)** 방식: "거리(홉 수)"와 "방향"만 보고 경로를 정합니다.
- **최대 홉 수 15** 제한이 있어 작은 네트워크에 적합합니다. (16홉이면 도달 불가로 간주)

비교: 또 다른 프로토콜 OSPF는 "링크 상태(Link State)" 방식이며 Dijkstra 알고리즘을 사용하고 큰 네트워크에 적합합니다.

암기 팁: "RIP = 홉 카운트, 거리 벡터, 최대 15홉".

19번

다음은 관계 대수에 대한 내용이다. 보기에 알맞는 기호를 작성하시오.

1. join
2. project
3. select
4. division

보기: $\bowtie$, σ , $\sqsubset$, π , $\sqsupset$, $\rhd$, $\div$

정답: (1) $\sqsupset$ ($\bowtie$) / (2) $\sqsubset$ (π) / (3) $\rhd$ (σ) / (4) $\rhd$ ($\div$)

해설:
관계 대수(Relational Algebra)의 연산 기호를 묻는 문제입니다. 기호와 의미를 외워야 합니다.

- **선택(Select) = σ (시그마):** 조건에 맞는 행(튜플)을 골라냅니다. 가로(행) 방향으로 자릅니다.
- **프로젝트(Project) = π (파이):** 원하는 열(속성)만 골라냅니다. 세로(열) 방향으로 자릅니다.
- **조인(Join) = $\bowtie$ (나비넥타이 모양):** 두 테이블을 공통 속성으로 연결합니다.
- **디비전(Division) = $\div$ (나누기):** 한 테이블에서 다른 테이블의 모든 값을 가진 행을 찾아냅니다.

암기 팁: "Select= σ (행 선택), Project= π (열 선택)". 시그마 σ 와 select, 파이 π 와 project를 글자 모양으로 연결해 외우면 좋습니다.

20번

다음은 데이터베이스에 관련된 문제이다. 괄호 안에 알맞는 답을 작성하시오.

- () 무결성은 2개의 관련 있는 관계 변수(테이블) 간의 일관성을 말한다.
- 기본 키 또는 후보 키와 외래 키의 조합으로 강제 적용된다.
- 외래 키 값은 부모 관계 변수의 기본 키 값 또는 후보 키 값으로 존재해야 한다.

정답: 참조

해설:

참조 무결성(Referential Integrity)을 묻는 문제입니다. 무결성이란 "데이터가 정확하고 일관된 상태를 유지하는 규칙"입니다.

- **참조 무결성**: 한 테이블의 **외래 키(Foreign Key)** 값은, 그것이 참조하는 다른 테이블의 **기본 키(또는 후보 키)에 실제로 존재하는 값**이어야 한다는 규칙입니다.
- 예: 주문 테이블의 "회원번호"(외래 키)는 반드시 회원 테이블에 존재하는 회원번호여야 합니다. 없는 회원이 주문을 할 순 없겠죠.

무결성 종류 정리:

- **개체 무결성**: 기본 키는 **NULL이 될 수 없고 중복도 안 됨**.
- **참조 무결성**: 외래 키는 **참조 대상에 존재하는 값이거나 NULL**이어야 함.
- **도메인 무결성**: 속성 값은 **정해진 범위(도메인)** 안의 값이어야 함.

키워드 "두 테이블 간 일관성, 외래 키"는 **참조 무결성**입니다.